

ADJ Question Bank — Solutions by Topic

TOPIC 1: Vector, ArrayList & HashSet

Q1 — A customer database contains duplicate customer IDs due to repeated registrations. Write a Java program using HashSet to remove duplicate IDs and display the unique customer IDs. (HashSet)

```
java

/* HashSet auto-rejects duplicates on add(). Add all IDs, print the set. */
import java.util.*;
public class Q1 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of customer IDs: ");
        int n = sc.nextInt();
        Set<Integer> s = new HashSet<>();
        System.out.println("Enter " + n + " customer IDs:");
        for (int i = 0; i < n; i++) s.add(sc.nextInt());
        System.out.println("Unique IDs: " + s);
    }
}
```

Q2 — A digital library stores book titles entered by users. Determine the number of unique titles available in the system using a HashSet. (HashSet)

```
java

/* Add titles to HashSet; size() gives unique count. */
import java.util.*;
public class Q2 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of book titles: ");
        int n = sc.nextInt(); sc.nextLine();
        Set<String> s = new HashSet<>();
        System.out.println("Enter " + n + " book titles:");
        for (int i = 0; i < n; i++) s.add(sc.nextLine());
        System.out.println("Unique titles: " + s);
        System.out.println("Count: " + s.size());
    }
}
```

Q3 — A password validation system requires all characters in a password to be unique. Write a Java program using HashSet to determine whether a given password contains duplicate characters. (HashSet)

```
java

/* Try adding each char; add() returns false if duplicate exists. */
import java.util.*;
public class Q3 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter password: ");
        String pwd = sc.next();
        Set<Character> s = new HashSet<>();
        boolean dup = false;
        for (char c : pwd.toCharArray())
            if (!s.add(c)) { dup = true; break; }
        System.out.println(dup ? "Duplicate chars found" : "All unique");
    }
}
```

Q4 — A customer feedback system needs to identify the first repeating character in a message. Implement the solution using HashSet. (HashSet)

```
java

/* HashSet.add() returns false if char already present – that is the first repeat. */
import java.util.*;
public class Q4 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter message: ");
        String msg = sc.nextLine();
        Set<Character> s = new HashSet<>();
        char rep = 0;
        for (char c : msg.toCharArray())
            if (!s.add(c)) { rep = c; break; }
        System.out.println(rep != 0 ? "First repeat: " + rep : "No repeat");
    }
}
```

Q5 — A messaging application wants to identify the first non-repeating character in a user message. Implement the solution using LinkedHashMap or suitable collection classes to preserve insertion order. (LinkedHashSet)

```

java

/* LinkedHashMap preserves insertion order. Count chars with Map, then scan LinkedHa
import java.util.*;
public class Q5 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter message: ");
        String msg = sc.nextLine();
        Map<Character,Integer> freq = new LinkedHashMap<>();
        for (char c : msg.toCharArray())
            freq.merge(c, 1, Integer::sum); // merge: if key absent put 1, else add
        char res = '?';
        for (char c : freq.keySet())
            if (freq.get(c) == 1) { res = c; break; }
        System.out.println("First non-repeating: " + res);
    }
}

```

Q6 — A plagiarism detection system needs to identify all unique words present in a document. Write a Java program using HashSet to display the unique words and count the total number of unique words. (HashSet)

```

java

/* Split paragraph by spaces, add each word to HashSet. */
import java.util.*;
public class Q6 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter document text: ");
        String doc = sc.nextLine();
        Set<String> s = new HashSet<>(Arrays.asList(doc.split(" ")));
        System.out.println("Unique words: " + s);
        System.out.println("Count: " + s.size());
    }
}

```

Q7 — A company maintains two sets of employee IDs. Write a Java program to determine whether one set is a subset of another using HashSet operations (HashSet)

java

```
/* containsAll() returns true if calling set contains every element of argument set.
import java.util.*;
public class Q7 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of IDs in set 1: ");
        int n1 = sc.nextInt();
        Set<Integer> s1 = new HashSet<>();
        System.out.println("Enter " + n1 + " IDs for set 1:");
        for (int i = 0; i < n1; i++) s1.add(sc.nextInt());
        System.out.print("Enter number of IDs in set 2: ");
        int n2 = sc.nextInt();
        Set<Integer> s2 = new HashSet<>();
        System.out.println("Enter " + n2 + " IDs for set 2:");
        for (int i = 0; i < n2; i++) s2.add(sc.nextInt());
        // containsAll: checks if s1 has every element of s2
        System.out.println("s2 subset of s1? " + s1.containsAll(s2));
    }
}
```

Q8 — Two departments maintain separate employee ID lists. Using HashSet, identify and display all common employee IDs. (HashSet intersection)

java

```
/* retainAll() keeps only elements present in both sets. */
import java.util.*;
public class Q8 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of IDs in department 1: ");
        int n1 = sc.nextInt();
        Set<Integer> d1 = new HashSet<>();
        System.out.println("Enter " + n1 + " IDs:");
        for (int i = 0; i < n1; i++) d1.add(sc.nextInt());
        System.out.print("Enter number of IDs in department 2: ");
        int n2 = sc.nextInt();
        Set<Integer> d2 = new HashSet<>();
        System.out.println("Enter " + n2 + " IDs:");
        for (int i = 0; i < n2; i++) d2.add(sc.nextInt());
        Set<Integer> common = new HashSet<>(d1);
        common.retainAll(d2); // retainAll: keep intersection only
        System.out.println("Common IDs: " + common);
    }
}
```


Q9 — A company maintains two departments' employee records. Using HashSet, display:

- **Employee IDs common to both departments**
- **Employee IDs present only in the first department**
- **Employee IDs present only in the second department**

java

```
/* retainAll=intersection, removeAll=difference, addAll=union. */
import java.util.*;
public class Q9 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of IDs in department 1: ");
        int n1 = sc.nextInt();
        Set<Integer> d1 = new HashSet<>();
        System.out.println("Enter " + n1 + " IDs:");
        for (int i = 0; i < n1; i++) d1.add(sc.nextInt());
        System.out.print("Enter number of IDs in department 2: ");
        int n2 = sc.nextInt();
        Set<Integer> d2 = new HashSet<>();
        System.out.println("Enter " + n2 + " IDs:");
        for (int i = 0; i < n2; i++) d2.add(sc.nextInt());

        Set<Integer> common = new HashSet<>(d1);
        common.retainAll(d2);
        System.out.println("Common: " + common);

        Set<Integer> only1 = new HashSet<>(d1);
        only1.removeAll(d2); // removeAll: remove elements present in d2
        System.out.println("Only in d1: " + only1);

        Set<Integer> only2 = new HashSet<>(d2);
        only2.removeAll(d1);
        System.out.println("Only in d2: " + only2);
    }
}
```

TOPIC 2: Stack, Queue & Deque

Q1 — Design a generic Stack<T> class that supports push, pop, and peek operations. Demonstrate its working using Integer values.

```

java

/* Generic Stack<T> wraps ArrayList. push=add end, pop=remove end, peek=read end. */
import java.util.*;
public class Q1 {
    static class Stack<T> {
        List<T> data = new ArrayList<>();
        void push(T v) { data.add(v); }
        T pop() { return data.isEmpty() ? null : data.remove(data.size()-1); }
        T peek() { return data.isEmpty() ? null : data.get(data.size()-1); }
        boolean empty() { return data.isEmpty(); }
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        Stack<Integer> s = new Stack<>();
        System.out.print("Enter number of integers to push: ");
        int n = sc.nextInt();
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++) s.push(sc.nextInt());
        System.out.println("Peek: " + s.peek());
        System.out.println("Pop: " + s.pop());
        System.out.println("Pop: " + s.pop());
    }
}

```

Q2 — Implement a generic Stack<T> class with push, pop, and peek operations. Demonstrate its working using String values.

java

```
/* Same Stack<T> implementation; demonstrated with String type. */
import java.util.*;
public class Q2 {
    static class Stack<T> {
        List<T> data = new ArrayList<>();
        void push(T v) { data.add(v); }
        T pop() { return data.isEmpty() ? null : data.remove(data.size()-1); }
        T peek() { return data.isEmpty() ? null : data.get(data.size()-1); }
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        Stack<String> s = new Stack<>();
        System.out.print("Enter number of strings to push: ");
        int n = sc.nextInt(); sc.nextLine();
        System.out.println("Enter " + n + " strings:");
        for (int i = 0; i < n; i++) s.push(sc.nextLine());
        System.out.println("Peek: " + s.peek());
        System.out.println("Pop: " + s.pop());
        System.out.println("Pop: " + s.pop());
    }
}
```

TOPIC 3: HashSet, LinkedHashSet, TreeSet

Q1 — A university maintains student ranks in sorted order. Store the ranks in a TreeSet, automatically eliminate duplicates, and display the ranks in ascending order.

java

```
/* TreeSet sorts automatically and rejects duplicates. */
import java.util.*;
public class Q1 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of ranks: ");
        int n = sc.nextInt();
        TreeSet<Integer> ts = new TreeSet<>();
        System.out.println("Enter " + n + " ranks:");
        for (int i = 0; i < n; i++) ts.add(sc.nextInt());
        System.out.println("Sorted unique ranks: " + ts);
    }
}
```

Q2 — A university stores student ranks in a TreeSet. Given a rank entered by the user, determine and display the nearest lower rank and nearest higher rank using TreeSet methods.

```
java

/* lower(x): greatest elem < x. higher(x): smallest elem > x. */
import java.util.*;
public class Q2 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of ranks: ");
        int n = sc.nextInt();
        TreeSet<Integer> ts = new TreeSet<>();
        System.out.println("Enter " + n + " ranks:");
        for (int i = 0; i < n; i++) ts.add(sc.nextInt());
        System.out.print("Enter rank to search: ");
        int r = sc.nextInt();
        System.out.println("Nearest lower than " + r + ": " + ts.lower(r));
        System.out.println("Nearest higher than " + r + ": " + ts.higher(r));
    }
}
```

Q3 — An online shopping application contains duplicate product IDs due to repeated imports from multiple vendors. Use a TreeSet to remove duplicates and display the product IDs in sorted order.

```
java

/* TreeSet: sorted + no duplicates. Simpler than HashSet when order matters. */
import java.util.*;
public class Q3 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of product IDs: ");
        int n = sc.nextInt();
        TreeSet<Integer> ts = new TreeSet<>();
        System.out.println("Enter " + n + " product IDs:");
        for (int i = 0; i < n; i++) ts.add(sc.nextInt());
        System.out.println("Sorted unique product IDs: " + ts);
    }
}
```

TOPIC 4: Generics — Basics

Q1 — Develop a generic method isEqualTo() that compares two objects and returns true if they are equal. Demonstrate the method using Integer and String values.

java

```
/* Single generic method comparing any two objects via equals(). */
import java.util.*;
public class Q1 {
    static <T> boolean isEqualTo(T a, T b) { return a.equals(b); }
    public static void main(String[] x) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter two integers: ");
        int i1 = sc.nextInt(), i2 = sc.nextInt(); sc.nextLine();
        System.out.println(isEqualTo(i1, i2));
        System.out.print("Enter two strings: ");
        String s1 = sc.next(), s2 = sc.next();
        System.out.println(isEqualTo(s1, s2));
    }
}
```

Q2 — GWrite a generic method `countOccurrences()` that counts the number of times a specified element appears in an array. Demonstrate it using Integer arrays.

java

```
/* Traverse array; equals() compares each element to target. */
import java.util.*;
public class Q2 {
    static <T> int count(T[] arr, T t) {
        int c = 0;
        for (T x : arr) if (t.equals(x)) c++;
        return c;
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        Integer[] nums = new Integer[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++) nums[i] = sc.nextInt();
        System.out.print("Enter target to count: ");
        int target = sc.nextInt();
        System.out.println("Count of " + target + ": " + count(nums, target));
    }
}
```

Q3 — A student record system needs to store a student's name and roll number together. Implement a generic `Pair<T,U>` class with methods to store and retrieve both values.

java

```
/* Generic pair class holds two values of different types. */
import java.util.*;
public class Q3 {
    static class Pair<T,U> {
        T fst; U snd;
        Pair(T f, U s) { fst = f; snd = s; }
        T get1() { return fst; }
        U get2() { return snd; }
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter student name: ");
        String name = sc.nextLine();
        System.out.print("Enter roll number: ");
        int roll = sc.nextInt();
        Pair<String,Integer> p = new Pair<>(name, roll);
        System.out.println("Name: " + p.get1() + ", Roll: " + p.get2());
    }
}
```

Q4 — Develop a generic Pair<T,U> class to store a product name and quantity. Include methods to set and retrieve the values and demonstrate its usage.

java

```
/* Same Pair class; demonstrated with String+Integer for product-qty. */
import java.util.*;
public class Q4 {
    static class Pair<T,U> {
        T fst; U snd;
        Pair(T f, U s) { fst=f; snd=s; }
        void set1(T v) { fst=v; } void set2(U v) { snd=v; }
        T get1() { return fst; } U get2() { return snd; }
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter product name: ");
        String name = sc.nextLine();
        System.out.print("Enter quantity: ");
        int qty = sc.nextInt();
        Pair<String,Integer> p = new Pair<>(name, qty);
        System.out.println(p.get1() + " -> qty: " + p.get2());
        System.out.print("Enter updated quantity: ");
        p.set2(sc.nextInt());
        System.out.println("Updated qty: " + p.get2());
    }
}
```

Q5 — Write a generic method `swapElements()` that swaps two elements of an array based on indices provided by the user.

java

```
/* Swap arr[i] and arr[j] using a temp variable. */
import java.util.*;
public class Q5 {
    static <T> void swap(T[] arr, int i, int j) {
        T tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        Integer[] nums = new Integer[n];
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++) nums[i] = sc.nextInt();
        System.out.print("Enter two indices to swap: ");
        int i = sc.nextInt(), j = sc.nextInt();
        swap(nums, i, j);
        for (int x : nums) System.out.print(x + " ");
    }
}
```

Q6 — A university stores students' CGPA values in arrays. Develop a generic method findMax() that accepts an array of any comparable type and returns the highest value. Demonstrate the method using both Integer and Double arrays.

java

```
/* Comparable bound ensures compareTo() is available on T. */
import java.util.*;
public class Q6 {
    static <T extends Comparable<T>> T findMax(T[] arr) {
        T m = arr[0];
        for (T x : arr) if (x.compareTo(m) > 0) m = x;
        return m;
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of integers: ");
        int n1 = sc.nextInt();
        Integer[] ints = new Integer[n1];
        System.out.println("Enter " + n1 + " integers:");
        for (int i = 0; i < n1; i++) ints[i] = sc.nextInt();
        System.out.print("Enter number of doubles: ");
        int n2 = sc.nextInt();
        Double[] dbls = new Double[n2];
        System.out.println("Enter " + n2 + " doubles:");
        for (int i = 0; i < n2; i++) dbls[i] = sc.nextDouble();
        System.out.println("Max int: " + findMax(ints));
        System.out.println("Max double: " + findMax(dbls));
    }
}
```

Q7 — A financial analysis application stores monthly profit values in a list. Develop a generic method findMin() that returns the smallest value from a list of comparable elements and demonstrate it using Double values.

java

```
/* Mirror of findMax; compareTo returns negative when x < m. */
import java.util.*;
public class Q7 {
    static <T extends Comparable<T>> T findMin(List<T> lst) {
        T m = lst.get(0);
        for (T x : lst) if (x.compareTo(m) < 0) m = x;
        return m;
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of profit values: ");
        int n = sc.nextInt();
        List<Double> profits = new ArrayList<>();
        System.out.println("Enter " + n + " profit values:");
        for (int i = 0; i < n; i++) profits.add(sc.nextDouble());
        System.out.println("Min profit: " + findMin(profits));
    }
}
```

Q8 — A university stores examination scores in Integer and Float arrays. Develop a generic sorting method that can sort arrays of different data types and display the sorted results.

java

```
/* Arrays.sort works on Comparable types; bound enforces this. */
import java.util.*;
public class Q8 {
    static <T extends Comparable<T>> void srt(T[] arr) {
        Arrays.sort(arr); // sort in natural ascending order
        System.out.println(Arrays.toString(arr));
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of integers: ");
        int n1 = sc.nextInt();
        Integer[] i = new Integer[n1];
        System.out.println("Enter " + n1 + " integers:");
        for (int k = 0; k < n1; k++) i[k] = sc.nextInt();
        System.out.print("Enter number of floats: ");
        int n2 = sc.nextInt();
        Float[] f = new Float[n2];
        System.out.println("Enter " + n2 + " floats:");
        for (int k = 0; k < n2; k++) f[k] = sc.nextFloat();
        srt(i); srt(f);
    }
}
```

Q9 — A financial application stores sales data in Integer and Float arrays. Develop a generic method `sumElements()` that accepts numeric arrays and calculates the sum of all elements. Demonstrate the method using both Integer and Float arrays.

```
java

/* T extends Number gives access to doubleValue() for summing any numeric type. */
import java.util.*;
public class Q9 {
    static <T extends Number> double sum(T[] arr) {
        double s = 0;
        for (T x : arr) s += x.doubleValue(); // doubleValue(): converts any Number
        return s;
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of integers: ");
        int n1 = sc.nextInt();
        Integer[] i = new Integer[n1];
        System.out.println("Enter " + n1 + " integers:");
        for (int k = 0; k < n1; k++) i[k] = sc.nextInt();
        System.out.print("Enter number of floats: ");
        int n2 = sc.nextInt();
        Float[] f = new Float[n2];
        System.out.println("Enter " + n2 + " floats:");
        for (int k = 0; k < n2; k++) f[k] = sc.nextFloat();
        System.out.println("Sum int:   " + sum(i));
        System.out.println("Sum float: " + sum(f));
    }
}
```

TOPIC 5: Advanced Generics — Wildcards

Q1 — A library stores lists of book titles and book IDs. Create a generic method `printList(List<?>)` using wildcards that can display elements from any type of list.

java

```
/* Unbounded wildcard <?> accepts list of any type; reads as Object. */
import java.util.*;
public class Q1 {
    static void printList(List<?> lst) {
        for (Object x : lst) System.out.print(x + " ");
        System.out.println();
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of book titles: ");
        int n1 = sc.nextInt(); sc.nextLine();
        List<String> titles = new ArrayList<>();
        System.out.println("Enter " + n1 + " titles:");
        for (int i = 0; i < n1; i++) titles.add(sc.nextLine());
        System.out.print("Enter number of book IDs: ");
        int n2 = sc.nextInt();
        List<Integer> ids = new ArrayList<>();
        System.out.println("Enter " + n2 + " IDs:");
        for (int i = 0; i < n2; i++) ids.add(sc.nextInt());
        printList(titles);
        printList(ids);
    }
}
```

Q2 — A library management system stores lists of book titles and book IDs. Create a generic method using wildcards to display elements from both types of lists.

```
java

/* Same unbounded wildcard pattern; works for any List type. */
import java.util.*;
public class Q2 {
    static void show(List<?> lst) {
        lst.forEach(System.out::println); // forEach: apply action to each element
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of book titles: ");
        int n1 = sc.nextInt(); sc.nextLine();
        List<String> titles = new ArrayList<>();
        System.out.println("Enter " + n1 + " titles:");
        for (int i = 0; i < n1; i++) titles.add(sc.nextLine());
        System.out.print("Enter number of book IDs: ");
        int n2 = sc.nextInt();
        List<Integer> ids = new ArrayList<>();
        System.out.println("Enter " + n2 + " IDs:");
        for (int i = 0; i < n2; i++) ids.add(sc.nextInt());
        show(titles);
        show(ids);
    }
}
```

Q3 — A warehouse management system maintains two lists: one containing product IDs and another containing inventory records. Develop a generic method `copyList()` using bounded wildcards to copy elements from one list to another.

java

```
/* <? extends T> (upper bound): safe to READ from src. Add to dest as T. */
import java.util.*;
public class Q3 {
    // src: producer (extends) – read only
    static <T> void copyList(List<? extends T> src, List<T> dest) {
        dest.addAll(src); // addAll: appends all elements of src to dest
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of product IDs: ");
        int n = sc.nextInt();
        List<Integer> src = new ArrayList<>();
        System.out.println("Enter " + n + " product IDs:");
        for (int i = 0; i < n; i++) src.add(sc.nextInt());
        List<Number> dest = new ArrayList<>();
        copyList(src, dest);
        System.out.println("Copied: " + dest);
    }
}
```

Q4 — Develop a generic Box<T> class that only accepts objects of type Number or its subclasses. Demonstrate the class using Integer and Double values.

java

```
/* Bounded class: T must be Number or subclass. Allows numeric methods inside. */
import java.util.*;
public class Q4 {
    static class Box<T extends Number> {
        T val;
        Box(T v) { val = v; }
        // doubleValue() available because T is guaranteed to be a Number
        double sqr() { return val.doubleValue() * val.doubleValue(); }
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter an integer value: ");
        int iv = sc.nextInt();
        System.out.print("Enter a double value: ");
        double dv = sc.nextDouble();
        Box<Integer> bi = new Box<>(iv);
        Box<Double> bd = new Box<>(dv);
        System.out.println(iv + " squared: " + bi.sqr());
        System.out.println(dv + " squared: " + bd.sqr());
    }
}
```

TOPIC 6: Swing GUI — JDialog Calculator

(No questions from bank map to pure Swing topic)

TOPIC 7: Multi-Frame GUI Wizard

(No questions from bank map to pure Swing wizard topic)

TOPIC 8: Reflection API

Q1 — A software company wants to identify classes that require security verification. Create a marker annotation called `@SecureClass`, apply it to a class, and use reflection to check whether the annotation is present.

```
java

/* Marker annotation has no members. isAnnotationPresent() checks at runtime. */
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME) // RUNTIME: annotation visible via reflection
@Target(ElementType.TYPE)           // TYPE: can be applied to classes
@interface SecureClass {}

@SecureClass
class BankService {}

class RegularService {}

public class Q1 {
    public static void main(String[] a) {
        // isAnnotationPresent: checks if class has the given annotation
        System.out.println("BankService secure? " + BankService.class.isAnnotationPresent(SecureClass.class));
        System.out.println("RegularService secure? " + RegularService.class.isAnnotationPresent(SecureClass.class));
    }
}
```

Q2 — A software development organization wants to mark classes that require auditing. Create a marker annotation called `@Audit`, apply it to appropriate classes, and use reflection to determine whether a class has been marked for auditing.

```

java

/* Same pattern as @SecureClass but named @Audit. */
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@interface Audit {}

@Audit class PayrollService {}
class InventoryService {}

public class Q2 {
    public static void main(String[] a) {
        System.out.println("PayrollService auditable? " + PayrollService.class.isAnnotationPresent(Audit.class));
        System.out.println("InventoryService auditable? " + InventoryService.class.isAnnotationPresent(Audit.class));
    }
}

```

Q3 — Create a single-value annotation called @CourseCode to store course information. Apply it to a class and retrieve its value using reflection.

```

java

/* Single-value annotation: one abstract method named value(). */
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@interface CourseCode {
    String value(); // single value – accessed as @CourseCode("ADJ")
}

@CourseCode("ADJ-2024")
class JavaCourse {}

public class Q3 {
    public static void main(String[] a) {
        // getAnnotation: retrieve annotation instance from class
        CourseCode cc = JavaCourse.class.getAnnotation(CourseCode.class);
        System.out.println("Course Code: " + cc.value());
    }
}

```

Q4 — Create a single-value annotation called @Subject to store subject information. Apply the annotation to a class and retrieve its value using reflection.


```
java

/* Same pattern as @CourseCode with a default value. */
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@interface Subject {
    String value() default "General"; // default: used when no value is supplied
}

@Subject("Advanced Java")
class AJCourse {}

@Subject // uses default
class BasicCourse {}

public class Q4 {
    public static void main(String[] a) {
        System.out.println(AJCourse.class.getAnnotation(Subject.class).value());
        System.out.println(BasicCourse.class.getAnnotation(Subject.class).value());
    }
}
```

Q5 — Create a single-value annotation with a default value, apply it to a class, and retrieve the annotation value using reflection.

```

java

/* Annotation with default value; retrieve without specifying value on @Tag. */
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@interface Tag {
    String value() default "untagged";
}

@Tag("production")
class ProdService {}

@Tag
class DevService {}

public class Q5 {
    public static void main(String[] a) {
        System.out.println(ProdService.class.getAnnotation(Tag.class).value());
        System.out.println(DevService.class.getAnnotation(Tag.class).value());
    }
}

```

Q6 — Create a multi-value annotation with default values to store employee information such as department, designation, and location. Retrieve and display the annotation values using reflection.

```

java

/* Multi-value: multiple abstract methods. Each acts as an annotation element. */
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@interface EmpInfo {
    String dept()    default "IT";
    String desig()   default "Engineer";
    String location() default "Pune";
}

@EmpInfo(dept="Finance", desig="Manager", location="Mumbai")
class FinanceEmp {}

public class Q6 {
    public static void main(String[] a) {
        EmpInfo e = FinanceEmp.class.getAnnotation(EmpInfo.class);
        System.out.println("Dept: "      + e.dept());
        System.out.println("Desig: "     + e.desig());
        System.out.println("Location: "  + e.location());
    }
}

```

Q7 — Create a multi-value annotation to store employee information such as ID, name, and department. Retrieve and display the values using reflection.

java

```
/* Multi-value annotation storing ID (int), name (String), dept (String). */
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@interface Employee {
    int id();
    String name();
    String dept();
}

@Employee(id=101, name="Rahul", dept="CS")
class Dev {}

public class Q7 {
    public static void main(String[] a) {
        Employee e = Dev.class.getAnnotation(Employee.class);
        System.out.println("ID: " + e.id());
        System.out.println("Name: " + e.name());
        System.out.println("Dept: " + e.dept());
    }
}
```

TOPIC 9: Lambda Expressions

Q1 — A mathematics learning application allows students to calculate the square of a number entered by the user. Create a functional interface and implement the square operation using a lambda expression.

```
java

/* @FunctionalInterface enforces exactly one abstract method. Lambda implements it.
import java.util.Scanner;
public class Q1 {
    @FunctionalInterface
    interface Sq { int apply(int x); }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        Sq sq = x -> x * x;
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        System.out.println("Square of " + n + ": " + sq.apply(n));
    }
}
```

Q2 — A text editor should calculate the total number of words in a paragraph entered by the user. Implement the solution using a lambda expression.

```
java

/* Split by whitespace; count tokens. */
import java.util.Scanner;
public class Q2 {
    @FunctionalInterface
    interface Counter { int count(String s); }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        Counter c = s -> s.trim().split("\\s+").length; // \\s+: one or more whitespace
        System.out.print("Enter a paragraph: ");
        String para = sc.nextLine();
        System.out.println("Words: " + c.count(para));
    }
}
```

Q3 — Develop a Java program that checks whether a given number is prime using a lambda expression.

```
java

/* Check divisibility from 2 to sqrt(n). Lambda wraps the logic. */
import java.util.Scanner;
public class Q3 {
    @FunctionalInterface
    interface Check { boolean test(int n); }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        Check isPrime = n -> {
            if (n < 2) return false;
            for (int i = 2; i <= Math.sqrt(n); i++)
                if (n % i == 0) return false;
            return true;
        };
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        System.out.println(n + " prime? " + isPrime.test(n));
    }
}
```

Q4 — Create a Java program that checks whether a word entered by the user is a palindrome using a block lambda expression.

java

```
/* Block lambda: multiple statements, explicit return. Reverses string and compares.
import java.util.Scanner;
public class Q4 {
    @FunctionalInterface
    interface Chk { boolean verify(String s); }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        Chk pal = s -> {
            String rev = new StringBuilder(s).reverse().toString();
            return s.equalsIgnoreCase(rev);
        };
        System.out.print("Enter a word: ");
        String word = sc.next();
        System.out.println(word + ": " + pal.verify(word));
    }
}
```

Q5 — A mathematics learning system allows students to calculate factorials of positive integers. Implement the factorial calculation using a block lambda expression and display the result.

java

```
/* Block lambda with loop computes n! iteratively. */
import java.util.Scanner;
public class Q5 {
    @FunctionalInterface
    interface Fact { long calc(int n); }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        Fact f = n -> {
            long r = 1;
            for (int i = 2; i <= n; i++) r *= i;
            return r;
        };
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        System.out.println(n + "! = " + f.calc(n));
    }
}
```

Q6 — A payroll system stores employee salaries in a list. Use lambda expressions to sort the salaries in both ascending and descending order and display the results.

```
java
```

```
/* Comparator lambda: (a,b)->a-b for asc, (a,b)->b-a for desc. */
import java.util.*;
public class Q6 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of salaries: ");
        int n = sc.nextInt();
        List<Integer> sal = new ArrayList<>();
        System.out.println("Enter " + n + " salaries:");
        for (int i = 0; i < n; i++) sal.add(sc.nextInt());
        sal.sort((x,y) -> x - y); // ascending
        System.out.println("Asc: " + sal);
        sal.sort((x,y) -> y - x); // descending
        System.out.println("Desc: " + sal);
    }
}
```

Q7 — A company maintains a list of employee names collected from different branches. The management wants the names to be displayed in alphabetical order for generating a company directory. Write a Java program using lambda expressions and Collections.sort() to sort and display the employee names.

```
java
/* String.compareTo gives lexicographic order; lambda wraps it. */
import java.util.*;
public class Q7 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of employee names: ");
        int n = sc.nextInt(); sc.nextLine();
        List<String> names = new ArrayList<>();
        System.out.println("Enter " + n + " names:");
        for (int i = 0; i < n; i++) names.add(sc.nextLine());
        Collections.sort(names, (x,y) -> x.compareTo(y)); // compareTo: alphabetic c
        System.out.println("Sorted: " + names);
    }
}
```

Q8 — A supermarket billing application allows the cashier to perform different arithmetic operations such as addition, subtraction, multiplication, and division on two item prices. Design a Java program using a functional interface and lambda expressions to perform the selected operation. Display the result based on the user's choice.

java

```
/* One functional interface; different lambdas assigned per operation. */
import java.util.Scanner;
public class Q8 {
    @FunctionalInterface
    interface Op { double apply(double a, double b); }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Op add = (a,b)->a+b, sub=(a,b)->a-b, mul=(a,b)->a*b, div=(a,b)->a/b;
        System.out.print("Enter two prices: ");
        double p1 = sc.nextDouble(), p2 = sc.nextDouble();
        System.out.println("Add: " + add.apply(p1, p2));
        System.out.println("Sub: " + sub.apply(p1, p2));
        System.out.println("Mul: " + mul.apply(p1, p2));
        System.out.println("Div: " + div.apply(p1, p2));
    }
}
```

Q9 — A scientific calculator application should perform addition or multiplication based on user selection. Implement the functionality using BiFunction and lambda expressions.

java

```
/* BiFunction<T,U,R>: takes two args, returns result. */
import java.util.*;
import java.util.function.*;
public class Q9 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        // BiFunction<input1, input2, output>
        BiFunction<Double,Double,Double> add = (x,y) -> x+y;
        BiFunction<Double,Double,Double> mul = (x,y) -> x*y;
        System.out.print("Enter two numbers: ");
        double x = sc.nextDouble(), y = sc.nextDouble();
        System.out.println("Add: " + add.apply(x, y));
        System.out.println("Mul: " + mul.apply(x, y));
    }
}
```

Q10 — An e-commerce application stores product names entered by sellers. Use Function<T,R> and lambda expressions to convert all product names into uppercase before displaying them.


```

java

/* Function<T,R>: single input T, single output R. */
import java.util.*;
import java.util.function.*;
public class Q10 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        Function<String,String> up = s -> s.toUpperCase();
        System.out.print("Enter number of products: ");
        int n = sc.nextInt(); sc.nextLine();
        System.out.println("Enter " + n + " product names:");
        for (int i = 0; i < n; i++) System.out.println(up.apply(sc.nextLine()));
    }
}

```

Q11 — A text analytics application needs to count how many times each word appears in a paragraph entered by the user. Use a Map and lambda expressions to display the frequency of each word.

```

java

/* merge(key, 1, Integer::sum): if absent put 1, else add 1 to existing count. */
import java.util.*;
public class Q11 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a paragraph: ");
        String para = sc.nextLine();
        Map<String,Integer> freq = new HashMap<>();
        for (String w : para.split(" "))
            freq.merge(w, 1, Integer::sum);
        freq.forEach((k,v) -> System.out.println(k + " -> " + v));
    }
}

```

TOPIC 10: Lambda — Checked Exceptions (Advanced Lambdas)

(Questions in bank don't specifically test *ThrowingConsumer* pattern — closest are covered in Topic 9)

TOPIC 11: Method References

Q1 — A utility class contains a static method that calculates the square of a number. Create a functional interface and invoke the method using a method reference instead of a lambda expression.

```
java

/* ClassName::staticMethod - no lambda wrapper needed. */
import java.util.*;
import java.util.function.*;
public class Q1 {
    static int sq(int x) { return x * x; }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        Function<Integer,Integer> f = Q1::sq; // static method ref
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        System.out.println("Square of " + n + ": " + f.apply(n));
    }
}
```

Q2 — A utility class contains a static method that calculates the square of a number. Create a functional interface and invoke the method using a static method reference.

```
java

/* Same static ref via a utility class. */
import java.util.*;
import java.util.function.*;
public class Q2 {
    static class Util { static int sq(int x) { return x*x; } }
    @FunctionalInterface interface Fn { int apply(int x); }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        Fn f = Util::sq; // static ref to Util.sq
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        System.out.println(f.apply(n));
    }
}
```

Q3 — A text-processing utility contains an instance method that checks whether a string is a palindrome. Use an instance method reference to invoke the palindrome-checking method.

```

java

/* obj::method – bound instance ref. The arg is passed to the method. */
import java.util.*;
import java.util.function.*;
public class Q3 {
    static class Checker {
        boolean isPalin(String s) {
            return s.equalsIgnoreCase(new StringBuilder(s).reverse().toString());
        }
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        Checker c = new Checker();
        Predicate<String> p = c::isPalin; // instance method ref on specific object
        System.out.print("Enter a word: ");
        String word = sc.next();
        System.out.println(word + ": " + p.test(word));
    }
}

```

Q4 — Create an Employee class with a parameterized constructor. Use constructor references to instantiate Employee objects.

```

java

/* ClassName::new maps to constructor. Supplier.get() creates the object. */
import java.util.*;
import java.util.function.*;
public class Q4 {
    static class Employee {
        String name;
        Employee(String n) { name = n; }
        void show() { System.out.println("Emp: " + name); }
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        Function<String,Employee> factory = Employee::new; // constructor ref
        System.out.print("Enter number of employees: ");
        int n = sc.nextInt(); sc.nextLine();
        System.out.println("Enter " + n + " employee names:");
        for (int i = 0; i < n; i++) factory.apply(sc.nextLine()).show();
    }
}

```

Q5 — A company wants to create Employee objects dynamically without explicitly calling constructors. Use constructor references and Supplier to create multiple Employee objects and display their details.

```

java

/* Supplier::get creates a new default object each call. */
import java.util.*;
import java.util.function.*;
public class Q5 {
    static class Employee {
        static int cnt = 0;
        int id; String name;
        Employee() { id=++cnt; name="Emp-"+id; }
        void show() { System.out.println(id+": "+name); }
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        Supplier<Employee> sup = Employee::new; // no-arg constructor ref
        System.out.print("Enter number of employees to create: ");
        int n = sc.nextInt();
        List<Employee> list = new ArrayList<>();
        for (int i=0;i<n;i++) list.add(sup.get()); // get(): creates new instance ea
        list.forEach(Employee::show); // instance method ref on arbitrary object (un
    }
}

```

Q6 — Create a class Student with a parameterized constructor. Use constructor references and a suitable functional interface to create Student objects dynamically.

```

java

/* BiFunction for parameterized constructor with two args. */
import java.util.*;
import java.util.function.*;
public class Q6 {
    static class Student {
        String name; int roll;
        Student(String n, int r) { name=n; roll=r; }
        void show() { System.out.println(roll+": "+name); }
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        BiFunction<String,Integer,Student> f = Student::new; // 2-arg constructor re
        System.out.print("Enter student name: ");
        String name = sc.next();
        System.out.print("Enter roll number: ");
        int roll = sc.nextInt();
        f.apply(name, roll).show();
    }
}

```

Q7 — Create an Employee class and store multiple Employee objects in a list. Use method references with `forEach()` to display employee names.

```
java

/* Unbound instance method ref: String::toUpperCase or Employee::show. */
import java.util.*;
public class Q7 {
    static class Employee {
        String name;
        Employee(String n) { name = n; }
        void show() { System.out.println(name); }
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of employees: ");
        int n = sc.nextInt(); sc.nextLine();
        List<Employee> list = new ArrayList<>();
        System.out.println("Enter " + n + " employee names:");
        for (int i = 0; i < n; i++) list.add(new Employee(sc.nextLine()));
        list.forEach(Employee::show); // forEach with unbound instance method ref
    }
}
```

Q8 — A customer relationship management system stores customer names in a list. Convert all names to uppercase using method references and Stream API, and display the resulting list.

```
java

/* String::toUpperCase: unbound ref – element becomes the object the method is called on */
import java.util.*;
import java.util.stream.*;
public class Q8 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of customer names: ");
        int n = sc.nextInt(); sc.nextLine();
        List<String> names = new ArrayList<>();
        System.out.println("Enter " + n + " names:");
        for (int i = 0; i < n; i++) names.add(sc.nextLine());
        List<String> upper = names.stream()
            .map(String::toUpperCase) // maps each string to its uppercase
            .collect(Collectors.toList()); // collect: gathers stream into a List
        System.out.println(upper);
    }
}
```

Q9 — A list of integers contains both even and odd values. Use method references with `Stream.filter()` to display only the even numbers.

```
java

/* Static method ref used as Predicate in filter. */
import java.util.*;
import java.util.stream.*;
public class Q9 {
    static boolean isEven(int n) { return n%2==0; }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of integers: ");
        int n = sc.nextInt();
        List<Integer> nums = new ArrayList<>();
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++) nums.add(sc.nextInt());
        nums.stream()
            .filter(Q9::isEven) // method ref as Predicate
            .forEach(System.out::println);
    }
}
```

Q10 — Create a generic static method that reverses an array. Invoke the method using a method reference and demonstrate it with Integer and String arrays.

java

```
/* Collections.reverse works on List; convert array, reverse, print. */
import java.util.*;
public class Q10 {
    static <T> void rev(T[] arr) {
        List<T> lst = Arrays.asList(arr);
        Collections.reverse(lst); // reverse(): reverses element order in-place
        System.out.println(lst);
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of integers: ");
        int n1 = sc.nextInt();
        Integer[] i = new Integer[n1];
        System.out.println("Enter " + n1 + " integers:");
        for (int k = 0; k < n1; k++) i[k] = sc.nextInt(); sc.nextLine();
        System.out.print("Enter number of strings: ");
        int n2 = sc.nextInt(); sc.nextLine();
        String[] s = new String[n2];
        System.out.println("Enter " + n2 + " strings:");
        for (int k = 0; k < n2; k++) s[k] = sc.nextLine();
        rev(i); rev(s);
    }
}
```

Q11 — A logistics company stores product information in two separate supplier lists. Merge the lists and display the combined result using Stream API.

```
java

/* Stream.concat joins two streams into one sequential stream. */
import java.util.*;
import java.util.stream.*;
public class Q11 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of items in list 1: ");
        int n1 = sc.nextInt(); sc.nextLine();
        List<String> l1 = new ArrayList<>();
        System.out.println("Enter " + n1 + " items:");
        for (int i = 0; i < n1; i++) l1.add(sc.nextLine());
        System.out.print("Enter number of items in list 2: ");
        int n2 = sc.nextInt(); sc.nextLine();
        List<String> l2 = new ArrayList<>();
        System.out.println("Enter " + n2 + " items:");
        for (int i = 0; i < n2; i++) l2.add(sc.nextLine());
        List<String> merged = Stream.concat(l1.stream(), l2.stream()) // concat: mer
            .collect(Collectors.toList());
        System.out.println("Merged: " + merged);
    }
}
```

TOPIC 12: Stream API

Q1 — A school management system stores student names in a list. Using Java Stream API, convert the list into another list containing the length of each student's name and display the result.

java

```
/* map transforms each element; collect gathers results into a list. */
import java.util.*;
import java.util.stream.*;
public class Q1 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of student names: ");
        int n = sc.nextInt(); sc.nextLine();
        List<String> names = new ArrayList<>();
        System.out.println("Enter " + n + " names:");
        for (int i = 0; i < n; i++) names.add(sc.nextLine());
        List<Integer> lens = names.stream()
            .map(String::length)
            .collect(Collectors.toList());
        System.out.println(lens);
    }
}
```

Q2 — A banking application maintains daily transaction amounts. Use Stream API to calculate and display the average transaction amount.

```
java

/* mapToDouble converts Stream<T> to DoubleStream for numeric operations. */
import java.util.*;
import java.util.stream.*;
public class Q2 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of transactions: ");
        int n = sc.nextInt();
        List<Double> tx = new ArrayList<>();
        System.out.println("Enter " + n + " transaction amounts:");
        for (int i = 0; i < n; i++) tx.add(sc.nextDouble());
        double avg = tx.stream()
            .mapToDouble(Double::doubleValue) // mapToDouble: creates primitive DoubleStream
            .average()                       // average(): returns OptionalDouble
            .orElse(0);
        System.out.printf("Average: %.2f%n", avg);
    }
}
```

Q3 — A sales management system stores daily sales figures in a list. Use Stream API to determine and display both the maximum and minimum sales values.

```

java

/* max/min return Optional; Comparator.naturalOrder() for ascending comparison. */
import java.util.*;
import java.util.stream.*;
public class Q3 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of sales figures: ");
        int n = sc.nextInt();
        List<Integer> sales = new ArrayList<>();
        System.out.println("Enter " + n + " sales values:");
        for (int i = 0; i < n; i++) sales.add(sc.nextInt());
        int max = sales.stream().max(Comparator.naturalOrder()).get(); // naturalOrd
        int min = sales.stream().min(Comparator.naturalOrder()).get();
        System.out.println("Max: "+max+", Min: "+min);
    }
}

```

Q4 — A university wants to calculate the total marks obtained by students. Use Stream reduce() to compute the sum of marks stored in a list.

```

java

/* reduce(identity, accumulator): folds stream to single value. */
import java.util.*;
public class Q4 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of students: ");
        int n = sc.nextInt();
        List<Integer> marks = new ArrayList<>();
        System.out.println("Enter " + n + " marks:");
        for (int i = 0; i < n; i++) marks.add(sc.nextInt());
        int total = marks.stream()
            .reduce(0, Integer::sum); // Integer::sum: static method ref as BinaryOp
        System.out.println("Total marks: " + total);
    }
}

```

Q5 — A school wants to identify the second-highest score obtained in an examination. Use Stream API to determine and display the second-largest distinct value.

```

java

/* sorted desc + distinct + skip(1) + findFirst = second highest. */
import java.util.*;
import java.util.stream.*;
public class Q5 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of scores: ");
        int n = sc.nextInt();
        List<Integer> scores = new ArrayList<>();
        System.out.println("Enter " + n + " scores:");
        for (int i = 0; i < n; i++) scores.add(sc.nextInt());
        int second = scores.stream()
            .distinct() // distinct: removes duplicates
            .sorted(Comparator.reverseOrder()) // reverseOrder: descending order
            .skip(1) // skip: bypasses first N elements
            .findFirst()
            .orElse(-1);
        System.out.println("2nd highest: " + second);
    }
}

```

Q6 — A school maintains a list of student roll numbers and wants to generate a separate list containing only even roll numbers. Use Stream API to collect all even numbers into a new list.

```

java

/* filter keeps only elements satisfying the predicate. */
import java.util.*;
import java.util.stream.*;
public class Q6 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of roll numbers: ");
        int n = sc.nextInt();
        List<Integer> rolls = new ArrayList<>();
        System.out.println("Enter " + n + " roll numbers:");
        for (int i = 0; i < n; i++) rolls.add(sc.nextInt());
        List<Integer> even = rolls.stream()
            .filter(x -> x%2==0)
            .collect(Collectors.toList());
        System.out.println("Even rolls: " + even);
    }
}

```

Q7 — A student database contains duplicate roll numbers due to data entry errors. Use Stream API to remove duplicate roll numbers and display the unique values.

java

```
/* distinct() removes duplicates; sorted() arranges in order. */
import java.util.*;
import java.util.stream.*;
public class Q7 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of roll numbers: ");
        int n = sc.nextInt();
        List<Integer> rolls = new ArrayList<>();
        System.out.println("Enter " + n + " roll numbers:");
        for (int i = 0; i < n; i++) rolls.add(sc.nextInt());
        List<Integer> uniq = rolls.stream()
            .distinct()
            .sorted()
            .collect(Collectors.toList());
        System.out.println("Unique rolls: " + uniq);
    }
}
```

Q8 — A registration system contains invalid user entries represented by null and empty strings. Use Stream filter() to remove invalid entries and display only valid records.

java

```
/* filter with null check + empty check removes bad records. */
import java.util.*;
import java.util.stream.*;
public class Q8 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of user entries (type 'null' for null, empty
        int n = sc.nextInt(); sc.nextLine();
        List<String> users = new ArrayList<>();
        System.out.println("Enter " + n + " entries:");
        for (int i = 0; i < n; i++) {
            String line = sc.nextLine();
            users.add(line.equals("null") ? null : line);
        }
        List<String> valid = users.stream()
            .filter(s -> s != null && !s.isEmpty()) // keeps non-null, non-empty
            .collect(Collectors.toList());
        System.out.println("Valid: " + valid);
    }
}
```

Q9 — A movie ticket booking system stores available seat numbers in a list. The booking engine needs to identify the first available even-numbered seat. Write a Java program using Stream API to find and display the first even seat number.

```
java

/* filter narrows, findFirst() returns Optional of first match. */
import java.util.*;
import java.util.stream.*;
public class Q9 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of seats: ");
        int n = sc.nextInt();
        List<Integer> seats = new ArrayList<>();
        System.out.println("Enter " + n + " seat numbers:");
        for (int i = 0; i < n; i++) seats.add(sc.nextInt());
        int seat = seats.stream()
            .filter(x -> x%2==0)
            .findFirst()
            .orElse(-1);
        System.out.println("First even seat: " + seat);
    }
}
```

Q10 — A company maintains monthly sales records and wants to identify the minimum sales value recorded during the year. Develop a generic method findMin() that works with comparable data types and demonstrate it using a list of Double values.

```
java

/* min with naturalOrder comparator finds smallest element. */
import java.util.*;
public class Q10 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of sales records: ");
        int n = sc.nextInt();
        List<Double> sales = new ArrayList<>();
        System.out.println("Enter " + n + " sales values:");
        for (int i = 0; i < n; i++) sales.add(sc.nextDouble());
        double min = sales.stream()
            .min(Comparator.naturalOrder())
            .get();
        System.out.println("Min sales: " + min);
    }
}
```

Q11 — A school wants to identify the second-highest distinct score obtained by students in an examination. Use Stream API to determine and display the result.

```
java

/* Same pattern; here with double scores. */
import java.util.*;
import java.util.stream.*;
public class Q11 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of scores: ");
        int n = sc.nextInt();
        List<Integer> sc2 = new ArrayList<>();
        System.out.println("Enter " + n + " scores:");
        for (int i = 0; i < n; i++) sc2.add(sc.nextInt());
        int res = sc2.stream().distinct().sorted(Comparator.reverseOrder()).skip(1).
        System.out.println("2nd highest: " + res);
    }
}
```

TOPIC 13: Date/Time & Locale

Q1 — A registration portal allows users to optionally provide their date of birth. Use Optional<Date> to validate the input and display appropriate messages if the value is missing.

```
java

/* Optional.ofNullable handles null safely; orElse provides fallback message. */
import java.util.*;
public class Q1 {
    static void show(Optional<java.util.Date> dob) {
        // ifPresentOrElse: Java 9+ – calls first if present, second if empty
        System.out.println(dob.isPresent() ? "DOB: " + dob.get() : "DOB not provided");
    }
    public static void main(String[] a) {
        show(Optional.of(new java.util.Date()));
        show(Optional.empty());
    }
}
```

Q2 — A user registration portal allows users to optionally provide an email address. Use Java's Optional class to safely handle missing email values and display appropriate messages.

```

java

/* orElse returns default string when Optional is empty. */
import java.util.*;
public class Q2 {
    static void show(Optional<String> email) {
        System.out.println(email.orElse("Email not provided"));
    }
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter email (or press Enter to skip): ");
        String input = sc.nextLine().trim();
        show(input.isEmpty() ? Optional.empty() : Optional.of(input));
        show(Optional.ofNullable(null)); // ofNullable: wraps null as empty Optional
    }
}

```

Q3 — A user profile application allows users to optionally provide their phone number. Use Java's Optional class to handle missing phone numbers and display appropriate messages.

```

java

/* ifPresent: executes Consumer only if value is present. */
import java.util.*;
public class Q3 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter phone number (or press Enter to skip): ");
        String input = sc.nextLine().trim();
        Optional<String> ph1 = input.isEmpty() ? Optional.empty() : Optional.of(input);
        Optional<String> ph2 = Optional.empty();
        ph1.ifPresent(p -> System.out.println("Phone: " + p)); // ifPresent: runs if present
        System.out.println(ph2.orElse("Phone not provided"));
    }
}

```

Q4 — Create a program that displays salary and date values formatted according to different locales such as India, USA, and Germany.

```

java

/* SimpleDateFormat with Locale formats date in regional style. */
import java.util.*;
import java.text.*;
public class Q4 {
    public static void main(String[] a) {
        Date now = new Date();
        // SimpleDateFormat pattern + Locale controls month names, separators
        SimpleDateFormat in = new SimpleDateFormat("dd-MM-yyyy", new Locale("en", "IN"));
        SimpleDateFormat us = new SimpleDateFormat("MM/dd/yyyy", Locale.US);
        SimpleDateFormat de = new SimpleDateFormat("dd.MM.yyyy", Locale.GERMANY);
        System.out.println("India: " + in.format(now));
        System.out.println("USA: " + us.format(now));
        System.out.println("Germany: " + de.format(now));
    }
}

```

Q5 — A travel company wants to calculate and display the time difference between India and New York. Use `TimeZone`, `Calendar`, and date formatting classes to implement the solution.

```

java

/* Same pattern; Japan uses yyyy/MM/dd convention. */
import java.util.*;
import java.text.*;
public class Q5 {
    public static void main(String[] a) {
        Date now = new Date();
        SimpleDateFormat de = new SimpleDateFormat("dd.MM.yyyy HH:mm:ss", Locale.GERMANY);
        SimpleDateFormat jp = new SimpleDateFormat("yyyy/MM/dd HH:mm:ss", Locale.JAPAN);
        System.out.println("Germany: " + de.format(now));
        System.out.println("Japan: " + jp.format(now));
    }
}

```

Q6 — Create a program that displays salary and date values formatted according to different locales such as India, USA, and Germany.

java

```
/* NumberFormat.getCurrencyInstance(Locale) adds currency symbol for that locale. */
import java.util.*;
import java.text.*;
public class Q6 {
    public static void main(String[] a) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter salary amount: ");
        double sal = sc.nextDouble();
        // getCurrencyInstance: formats number with locale-specific currency symbol
        NumberFormat in = NumberFormat.getCurrencyInstance(new Locale("en", "IN"));
        NumberFormat us = NumberFormat.getCurrencyInstance(Locale.US);
        NumberFormat de = NumberFormat.getCurrencyInstance(Locale.GERMANY);
        System.out.println("India:    " + in.format(sal));
        System.out.println("USA:      " + us.format(sal));
        System.out.println("Germany: " + de.format(sal));
    }
}
```

Q7 — A travel company wants to calculate and display the time difference between India and New York. Use TimeZone, Calendar, and date formatting classes to implement the solution

```
java

/* getOffset(ms): returns zone offset from UTC in ms. Divide by 3_600_000 for hours.
import java.util.*;
import java.text.*;
public class Q7 {
    public static void main(String[] a) {
        Calendar cal = Calendar.getInstance();
        TimeZone ist = TimeZone.getTimeZone("Asia/Kolkata");
        TimeZone est = TimeZone.getTimeZone("America/New_York");
        long diff = Math.abs(ist
```